Title

MediAssist — AI Patient Assistant for MediCare General Hospital

Problem Statement

Domain:

Healthcare, Hospital Patient Services

Users:

Patients seeking hospital information and reception staff managing repetitive queries.

Problem:

MediCare General Hospital receives a high volume of daily helpline calls. Most of these calls are repetitive, involving questions about OPD timings, appointments, fees, and insurance. This situation results in inefficient use of staff time. The current chatbots do not have specific knowledge about the hospital, struggle to maintain context, and can give incorrect or unsafe answers.

Success Criteria:

- Accurately answers at least 10 domain-specific queries using retrieved context
- Achieves RAGAS faithfulness ≥ 0.75 across evaluation queries
- Correctly handles adversarial inputs without making errors
- Maintains multi-turn memory, such as recalling patient names
- Responds to emergency queries instantly without retrieval

Tool:

LangGraph-based RAG system with structured routing and optional LLM integration, selected for controlled, context-aware and safe response generation in a healthcare environment.

Solution / Features

MediAssist is a graph-based conversational AI agent built using LangGraph. Each patient query goes through a structured multi-node pipeline. The system handles memory, routes queries, retrieves information, invokes tools, generates answers, evaluates responses, and maintains state in a controlled order. This process ensures reliability and safety in responses.

Key Features

• RAG-based Answering:

The system generates answers strictly from context retrieved from a 12-document hospital knowledge base stored in ChromaDB. It avoids hallucination and provides a fallback response if the information is missing.

• Structured Routing:

Each query is classified into predefined paths (retrieve, tool, skip, emergency). This enables the system to make clear decisions instead of relying only on LLM reasoning.

• Conversation Memory:

With LangGraph's MemorySaver and session-based thread IDs, the system keeps track of multi-turn context. It can recall patient-specific details, like names, across different interactions.

• Tool Integration:

For real-time queries, such as the current time or date, the system uses a dedicated datetime tool rather than static knowledge. This ensures accurate, up-to-date responses.

• Self-Evaluation Loop:

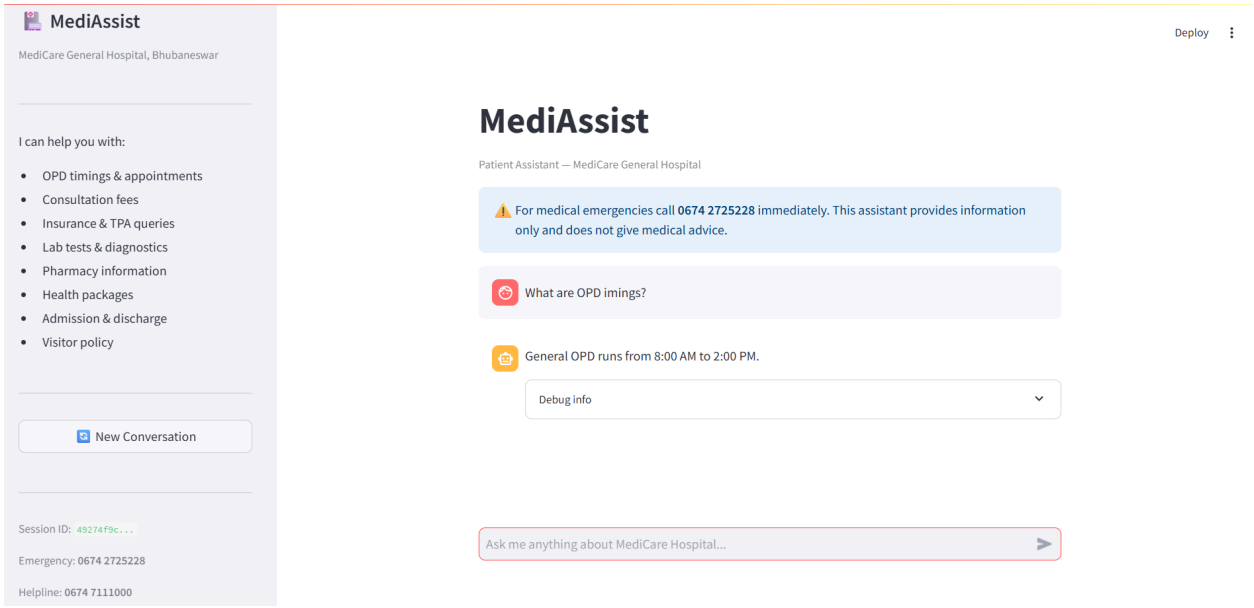
The system includes an evaluation node that checks if the generated response is based on retrieved context. If the score is below 0.7, it retries answer generation with stricter limits, which improves reliability.

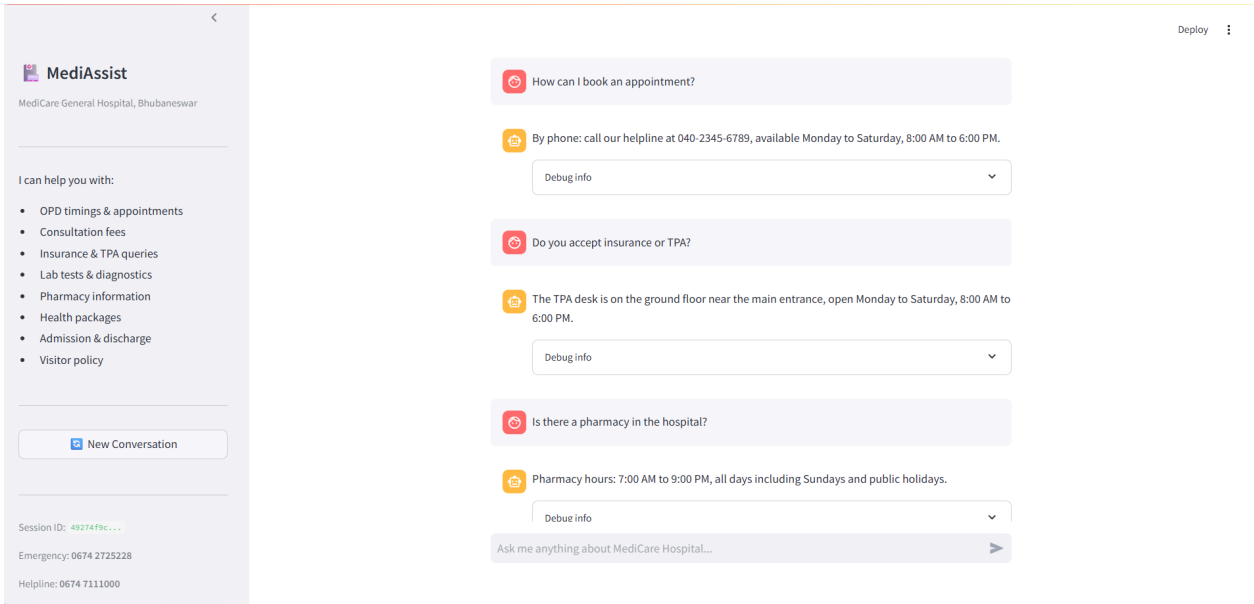
• Streamlit Interface:

The application is set up as an interactive web interface with chat features, session persistence, a reset option, and a debug panel that shows the route, faithfulness score, and source references.

Screenshots

The following screenshots demonstrate key functionalities including RAG-based answering, multi-turn memory, and the interactive Streamlit interface.





Tech Stack

- Programming Language: Python
- Framework: LangGraph (for agent workflow orchestration)
- LLM: Groq (LLaMA 3 model) [optional; system supports fallback mode]
- Vector Database: ChromaDB (in-memory storage)
- Embedding Model: SentenceTransformers (all-MiniLM-L6-v2)
- Frontend/UI: Streamlit
- Evaluation: RAGAS (faithfulness, relevancy, context precision)

Unique Features

• Strict Grounding Contract:

MediAssist enforces a grounding contract in the answer node's system prompt. This contract clearly stops the language model from using any knowledge from its training data when answering hospital-related questions. If the exact answer is not available in the retrieved context, the model is instructed to respond with a specific helpline redirect phrase. This approach is stronger than a typical RAG implementation that simply asks the model to “prefer” the context.

• Self-Reflection Eval Loop:

The faithfulness eval loop is not a post-processing filter. Instead, it is an integrated part of the graph that can route execution back to the answer node with modified instructions. This means the system actively tries to correct low-quality answers within a single invocation rather than just flagging them for human review.

• Red-team Validated Robustness:

The system underwent tests against five categories of adversarial input before submission. Test results show that it correctly handles prompt injection, hallucination bait, false premise acceptance, out-of-scope task completion, and missing information fabrication. This level of adversarial testing is not common in typical chatbot implementations.

• Transparent Debug Panel:

Every response includes a debug panel that reveals the internal routing decision, faithfulness score, and source topics to the end user. This transparency helps patients, staff, and evaluators understand why a specific answer was given and identify cases where the retrieval may have been imprecise.

Future Improvements

• Multilingual Support:

The current system operates in English only. Bhubaneswar's patient population primarily communicates in Odia and Hindi. Adding multilingual support with a model like paraphrase-multilingual-MiniLM-L12-v2 and a multilingual language model would greatly expand the system's reach without changing the graph architecture.

• Real Hospital System Integration:

The current knowledge base consists of static documents. In a production setting, these documents should be replaced with live data feeds from the hospital's management information system. This includes real-time doctor availability, current waiting times, and

live bed availability. To achieve this, a database connector tool needs to be added alongside the existing datetime tool.

• Appointment Booking System:
Phase 1 of MediAssist provides information only. Phase 2 should let patients book, cancel, and reschedule appointments directly through the chat interface by integrating with the hospital's appointment management API. This would involve adding a new tool node for appointment operations and expanding the router to recognize booking intent.

• Improved Evaluation Framework:
The current RAGAS evaluation uses five manually created QA pairs. A stronger evaluation framework would include a larger golden dataset of 50 to 100 verified question-answer pairs. It should run automated evaluations on every code change as part of a continuous integration process and track metric trends over time to spot regressions.

Conclusion

MediAssist shows that a production-oriented hospital patient assistant can be built using LangGraph. It features a clearly defined state machine, a verified knowledge base, and a faithfulness evaluation loop all within a single development cycle. The system accurately answers common hospital questions, keeps track of patient context across conversations, handles time-sensitive queries with a live tool, and does not create information it does not possess.

The architecture is modular, each node can be tested independently, the knowledge base can be expanded by adding documents without altering any node code, and the Streamlit interface can be switched out for a WhatsApp or FastAPI endpoint without modifying the graph. This makes MediAssist a practical starting point for a production deployment rather than just a proof-of-concept prototype.

In a real-world context, a system like this eases the burden on hospital helpline staff, gives patients instant and accurate answers at any hour, and upholds the trust and safety standards required in healthcare by never inventing information it does not have.

Name: Ananwita Dash
Roll Number: 2330067
Batch / Program: Agentic AI